Day 102: Question Answering with a Pretrained Transformer on a Real Dataset

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 5 | Complete study guide

Watch: Applying BERT to Question Answering (SQuAD v1.1) (InnerWorkingsAI) | Text Extraction From a Corpus Using BERT (AKA Question Answering) (Abhishek Thakur)

Script: `day102_question_answering.py` · every number in this guide is copied by program from `day102_run_output.txt`, the real output of that script.

A pretrained DistilBERT reads a question and a Wikipedia paragraph and points at the answer. On 1,000 real SQuAD questions it gets 0.8010 exact match and 0.8732 F1, with SQuAD's official metrics rewritten from scratch; the best-overlap sentence, a from-scratch baseline, contains the answer 0.8310 of the time but scores F1 0.1867. Given the wrong paragraph the model still answers, but its own score almost always shows it (ROC area 0.989) — the key fact for Day 104's retrieval system.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

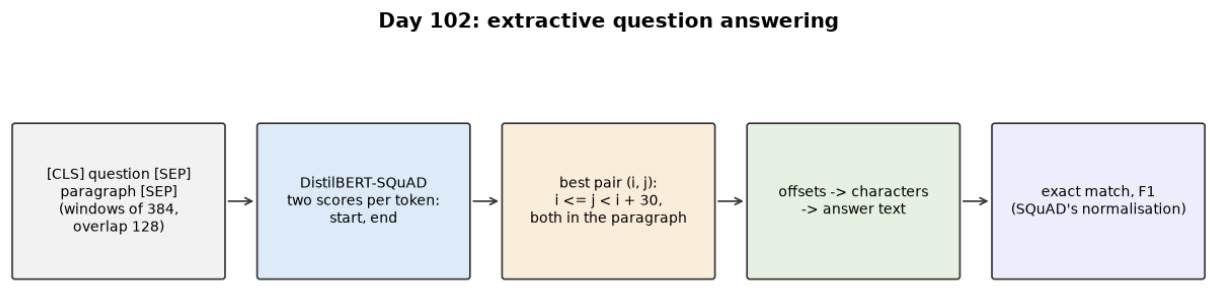
Extractive question answering: the answer is a span of the given paragraph's own text. The model reads '[CLS] question [SEP] paragraph [SEP]' and gives every token two scores: how likely it is to **start** the answer, and how likely to **end** it. The answer is the best (start, end) pair with start before end.

SQuAD v1.1: about 100,000 questions written by crowd workers about Wikipedia paragraphs; every question's answer is in its paragraph, and most have several acceptable answer strings.

REAL-WORLD SCENARIO

A reading-comprehension test: underline the words in the passage that answer the question.

SYSTEM DIAGRAM



Question and paragraph in, two scores per token, the best valid span out.

Today's part for Day 104

`day102_exercise.py` is the scoring part: `normalize`, `exact_match` and `f1`, SQuAD's official definitions rewritten and checked on cases worked out by hand. Its real output:

ACTUAL OUTPUT

```
prediction                          gold answers                    EM     F1   why
'The Denver Broncos'                ['Denver Broncos']               1  1.000   articles and case
    are ignored
'Denver Broncos.'                   ['Denver Broncos']               1  1.000   punctuation is
    ignored
'Broncos'                           ['Denver Broncos']               0  0.667   one of two words:
    precision 1, recall 0.5
'the Denver Broncos football team'  ['Denver Broncos']               0  0.667   two of four words:
    precision 0.5, recall 1
'Carolina Panthers'                 ['Denver Broncos', 'Broncos']    0  0.000   no shared word
'Broncos'                           ['Denver Broncos', 'Broncos']    1  1.000   matches the second
    gold answer exactly
'1,000'                             ['1000']                         1  1.000   the comma is
    punctuation: 1,000 becomes 1000
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. How the model answers

DEFINITION — start here, no prior knowledge assumed

The best span maximises start score + end score over pairs with start ≤ end < start + 30 tokens, both inside the paragraph. A paragraph too long for one 384-token input is cut into overlapping windows (overlap 128) and the best span over all windows wins. Token offsets map the span back to the paragraph's exact characters.

REAL-WORLD SCENARIO

Marking where an answer begins and where it ends, and refusing a 'start' that comes after the 'end'.

MATHEMATICS

$$(i^*, j^*) = \operatorname{argmax}_{i,j} s_i + e_j \quad \text{subject to } i \le j < i + 30, \text{ both in the paragraph}$$

IMPLEMENTATION

```
def answer(tok, model, question, context):
    enc = tok(question, context, truncation="only_second", max_length=MAX_LEN, stride=STRIDE,
              return_overflowing_tokens=True, return_offsets_mapping=True, padding=True,
                  return_tensors="pt")
    offsets = enc.pop("offset_mapping").numpy(); enc.pop("overflow_to_sample_mapping")
    with torch.no_grad():
        out = model(**enc)
    best = (-np.inf, "")
    for w in range(len(offsets)):
        seq = enc.sequence_ids(w)
        ctx = np.array([s == 1 for s in seq])
        st, en = out.start_logits[w].numpy(), out.end_logits[w].numpy()
        S = st[:, None] + en[None, :]
        ok = np.triu(np.ones_like(S, dtype=bool)) & ~np.triu(np.ones_like(S, dtype=bool),
            MAX_ANSWER)
        ok &= ctx[:, None] & ctx[None, :]
        S = np.where(ok, S, -np.inf)
        i, j = np.unravel_index(np.argmax(S), S.shape)
        if S[i, j] > best[0]:
            best = (S[i, j], context[offsets[w][i][0]:offsets[w][j][1]])
    return best[1], float(best[0])
```

ACTUAL OUTPUT

```
question: What did Low suggest using Apollo 8 for instead of simply orbiting the Earth's surface?
paragraph (174 words, 241 tokens with the question): Apollo 8 was planned to be the D mission in December
    1968, crewed by McDivitt, Scott and Schweickart, launched on a Saturn V instead of two Saturn IBs. In the
    s...
highest start scores: 'eliminating' -0.86, 'def' -0.88, 'bold' -1.72
highest end scores:   'mission' 1.24, 'track' -0.69, '.' -1.43
best valid span: 'eliminating the E mission' (start + end score 0.38); gold answers: ['orbit the Moon', 'orbit
    the Moon', 'to orbit the Moon', 'orbit the Moon', 'orbit the Moon']
(the library's question-answering pipeline was removed in transformers 5; installed: 5.22.0; so the span
    search is checked against the naive alternative instead)
naive search (best start and best end chosen separately) gives an invalid span for 6 of the first 200
    questions; the constrained search never does
question + paragraph longer than 384 tokens (needs more than one window): 18 of 1000
Actual output from this lesson's run.
```

What it means

The very first question shows the mechanics and a real mistake: the best start and end form 'eliminating the E mission', while every annotator wrote 'orbit the Moon'. Choosing the start and end separately, as a naive search would, gave an invalid span (end before start, too long, or in the question) for 6 of the first 200 questions; the constrained search cannot. Only 18 of 1000 question-paragraph pairs needed more than one window. A note on the library: transformers 5 (installed here) removed its question-answering pipeline, so this span search is not just an exercise — it is how the model has to be used now.

WHY CHOOSE THIS?

Answers are always real text from the paragraph, with exact character positions.

WHY NOT THIS? (TRADEOFFS)

It can only copy: no yes/no answers, no combining facts from two places.

2. SQuAD's metrics: exact match and F1

DEFINITION — start here, no prior knowledge assumed

Both compare a prediction with every acceptable answer and keep the best. **Exact match** is 1 if they are identical after normalising (lowercase, no punctuation, no 'a/an/the'). **F1** gives partial credit for shared words: 2PR/(P + R) with P = shared/predicted words and R = shared/gold words.

REAL-WORLD SCENARIO

Marking a short answer: exactly right earns the mark; 'nearly right' earns part of it.

MATHEMATICS

$$EM = \max_g [\text{norm}(pred) = \text{norm}(g)], \quad F1 = \max_g 2PR/(P + R)$$

IMPLEMENTATION

```
def normalize(s):
    s = "".join(ch for ch in s.lower() if ch not in set(string.punctuation))
    s = re.sub(r"\b(a|an|the)\b", " ", s)
    return " ".join(s.split())
def exact_match(pred, golds):
    return float(any(normalize(pred) == normalize(g) for g in golds))
def f1(pred, golds):
    def one(p, g):
        p, g = normalize(p).split(), normalize(g).split()
        common = sum((Counter(p) & Counter(g)).values())
        if common == 0:
            return 0.0
        prec, rec = common / len(p), common / len(g)
        return 2 * prec * rec / (prec + rec)
    return max(one(pred, g) for g in golds)
```

ACTUAL OUTPUT

```
1,000 questions drawn at random from 10,570 (114 ms per question, 4 threads); 3.28 gold answers per question
  exact match  0.8010   95% interval [0.7760, 0.8260]
  F1           0.8732   95% interval [0.8542, 0.8914]
examples where F1 gives partial credit (exact match 0, F1 above 0):
  predicted 'future angel-like life in heaven in the midst of this life' gold 'service to the neighbor'
      F1 0.286
  predicted 'Pen-y-darren ironworks, near Merthyr Tydfil to Abercynon in south Wales' gold 'Abercynon'
      F1 0.571
  predicted 'to continue to cut production in five percent monthly increments until their economic and
      political objectives were met' gold 'until their economic and political objectives were met' F1 0.615
  predicted 'Subject Committees in the fourth Session' gold 'Session'                          F1 0.333
Actual output from this lesson's run.
```

What it means

Exact match 0.8010 and F1 0.8732, each about two points either way. F1's partial credit is visible in the examples: the model's answers are often a little longer than the annotators' ('Pen-y-darren ironworks, ... to Abercynon' for 'Abercynon', F1 0.571). With 3.28 gold answers per question, a reasonable answer usually matches at least one.

WHY CHOOSE THIS?

Standard, reproducible, and generous to reasonable variations through multiple gold answers.

WHY NOT THIS? (TRADEOFFS)

Word overlap only: a correct paraphrase ('the Moon' vs 'Earth's satellite') scores 0.

3. A from-scratch baseline: the best-overlap sentence

DEFINITION — start here, no prior knowledge assumed

Pick the paragraph sentence (Day 101's splitter) sharing the most non-stop words with the question, and answer with the whole sentence.

REAL-WORLD SCENARIO

Skimming for the sentence that repeats the question's words: you find the right place but not the answer.

IMPLEMENTATION

```
def overlap_sentence(question, context):
    stop = {"the", "a", "an", "of", "in", "to", "is", "was", "what", "which", "who", "when",
        "where", "how", "why", "did", "does",
            "do", "and", "for", "on", "by", "with", "as", "at", "that", "from", "are", "were",
                "it", "its"}
    q = {w for w in re.findall(r"\w+", question.lower()) if w not in stop}
    sents = d101.split_sentences(context) or [context]
    return max(sents, key=lambda s: len(q & set(re.findall(r"\w+", s.lower()))))
```

ACTUAL OUTPUT

```
the chosen sentence contains a gold answer: 0.8310
answering with the whole sentence: exact match 0.0040, F1 0.1867 (mean sentence 30.6 words against 3.0 for a
    gold answer)
model minus baseline, F1: +0.6868 [+0.6663, +0.7072] (Day 99's paired bootstrap)
Actual output from this lesson's run.
```

What it means

Word overlap finds the right sentence 0.8310 of the time — SQuAD's questions reuse the paragraph's words (Day 103 measures this) — but a 30.6-word sentence is a poor answer to a 3.0-word question: F1 0.1867. The model's advantage is +0.6868 F1: the hard part is the exact span.

WHY CHOOSE THIS?

Shows what locating alone is worth.

WHY NOT THIS? (TRADEOFFS)

A stronger baseline would extract a phrase from the sentence; not built.

4. Where it fails: question types

DEFINITION — start here, no prior knowledge assumed

Accuracy by the question word.

REAL-WORLD SCENARIO

A student who is sure about names and dates but vague about explanations.

ACTUAL OUTPUT

```
type       questions  exact match      F1
what             564        0.785   0.855
who              109        0.890   0.918
when              85        0.859   0.911
which             59        0.780   0.863
how many          53        0.868   0.925
how               49        0.633   0.813
where             43        0.837   0.909
other             15        0.867   0.960
how much          14        0.786   0.874
why                9        0.556   0.894
Actual output from this lesson's run.
```

What it means

'Who' (0.890) and 'how many' (0.868) questions have short, well-defined answers; 'how' (0.633) and 'why' (0.556) questions ask for descriptions with no single right boundary, so exact match suffers while F1 stays higher (0.813, 0.894). Only 9 'why' questions: too few to rank.

WHY CHOOSE THIS?

Points to where the boundary, not the understanding, is the problem.

WHY NOT THIS? (TRADEOFFS)

Small groups; the first matching question word is a crude type.

5. The wrong paragraph: does the score know?

DEFINITION — start here, no prior knowledge assumed

The first 300 questions again, each with the paragraph of a question from a different article. A SQuAD v1.1 model must still answer. The question is whether its score (start + end) separates right from wrong paragraphs; the **ROC area** is the chance that a random right-paragraph answer scores higher than a random wrong-paragraph one.

REAL-WORLD SCENARIO

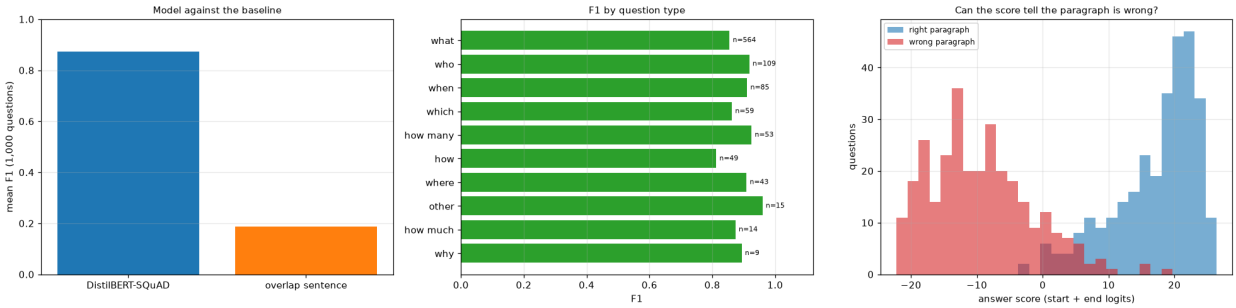
A witness who always answers: you need to know when they are guessing.

IMPLEMENTATION

```
titles = [q["title"] for q in Q]
wrong = []
for i, q in enumerate(Q[:300]):
    j = next(k for k in range(i + 1, i + len(Q)) if titles[k % len(Q)] != titles[i])
    wrong.append(answer(tok, model, q["question"], Q[j % len(Q)]["context"]))
right_s = np.array([s for _, s in preds[:300]]); wrong_s = np.array([s for _, s in wrong])
print(f"\nscore (start + end) with the right paragraph: median {np.median(right_s):.2f}; with a
    wrong one: median {np.median(wrong_s):.2f}")
print("sample of answers from the wrong paragraph: " + "; ".join(f"{q['question'][:40]!r} ->
    {t[:30]!r}" for q, (t, _) in list(zip(Q, wrong))[:3]))
pos = np.r_[right_s, wrong_s]; lab = np.r_[np.ones(300), np.zeros(300)]
order = np.argsort(-pos); tp = np.cumsum(lab[order]); fp = np.cumsum(1 - lab[order])
auc = np.trapezoid(np.r_[0, tp / 300], np.r_[0, fp / 300])
```

ACTUAL OUTPUT

```
The first 300 questions again, each paired with the paragraph of a question from a DIFFERENT article. SQuAD
    v1.1 models were never trained to say 'no answer here', so they always point at some span.
score (start + end) with the right paragraph: median 19.26; with a wrong one: median -10.23
sample of answers from the wrong paragraph: 'What did Low suggest using Apollo 8 for ' -> 'poor physical
    health'; "How did Luther's writings sound as he be" -> 'less sucrose'; 'When might starch grains become
    overly l' -> 'during the Nazi Third Reich'
area under the ROC curve, score as a 'right paragraph' detector: 0.989 (0.5 = useless, 1 = perfect)
  refuse below 2.0: keeps 0.967 of right-paragraph answers (their exact match 0.807), lets through 0.090 of
      wrong-paragraph ones
  refuse below 5.0: keeps 0.940 of right-paragraph answers (their exact match 0.816), lets through 0.050 of
      wrong-paragraph ones
  refuse below 8.0: keeps 0.880 of right-paragraph answers (their exact match 0.830), lets through 0.023 of
      wrong-paragraph ones
Actual output from this lesson's run.
```



Left: F1 of the model and the baseline. Middle: F1 by question type. Right: answer scores with the right and the wrong paragraph.

What it means

With the wrong paragraph the model answers confidently-sounding nonsense ('poor physical health', 'less sucrose'), but its score drops from a median 19.26 to -10.23, and the ROC area is 0.989. A threshold of 5 keeps 0.940 of right-paragraph answers and lets through only 0.050 of wrong-paragraph ones. Wrong paragraphs from other articles are an easy case; a wrong paragraph from the right article would be harder (not tested).

WHY CHOOSE THIS?

A free 'I don't know' signal for a retrieval system.

WHY NOT THIS? (TRADEOFFS)

Thresholds must be chosen on separate data; this test used only unrelated paragraphs.

Interview preparation — technical questions

Q: How does a BERT-style model answer extractive questions?

A: Start and end scores per token; the best valid pair (start <= end, limited length, inside the paragraph).

Q: How are long paragraphs handled?

A: Overlapping windows with the full question in each; the best span over all windows.

Q: What are SQuAD's EM and F1?

A: Exact match after normalising; word-overlap F1; both the best over the gold answers.

Q: Why not pick the best start and best end separately?

A: They can form an invalid span: 6 of 200 questions here.

Q: How can a QA model say 'not here'?

A: Threshold its own score; here ROC area 0.989 against unrelated paragraphs.

Interview preparation — theoretical questions

Q: Why is F1 higher than exact match?

A: Boundaries are a matter of judgement; partial overlap earns partial credit.

Q: Why are 'why' and 'how' questions harder?

A: Their answers are descriptions without a single natural boundary.

Q: What is SQuAD 2.0's change?

A: It adds unanswerable questions, so models must learn to abstain.

Q: What does extractive QA need from a retriever?

A: A passage that contains the answer: Day 104.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Extractive QA is two scores per token and a constrained search for the best span. On real SQuAD questions it reaches 0.8010 exact match and 0.8732 F1, far beyond finding the right sentence by word overlap (F1 0.1867). It always answers, but its score reliably drops when the paragraph is wrong: the hook for a retrieval system.

- EM 0.8010 [0.7760, 0.8260], F1 0.8732 [0.8542, 0.8914] on 1,000 questions.
- Baseline: right sentence 0.8310 of the time, F1 0.1867.
- Hardest: 'how' (EM 0.633) and 'why' (EM 0.556, only 9 questions).
- Wrong paragraph: median score 19.26 -> -10.23, ROC area 0.989.

Suggested watch order, if starting from zero

- 1. Applying BERT to Question Answering (SQuAD v1.1) (InnerWorkingsAI) — how BERT's start and end scores answer SQuAD.
- 2. Text Extraction From a Corpus Using BERT (AKA Question Answering) (Abhishek Thakur) — the same idea as text extraction, with code.

Day 103 preview

- **Semantic search.** Today the right paragraph was given. Tomorrow it has to be found among 2,067, by keyword search (BM25, from scratch) and by sentence embeddings.